Compte rendu

Ouarezki Oussama Abde Rahim G3

Introducation

In this practical session, we will predict salary based on years of experience using linear and polynomial regression models. By visualizing the data, we will determine which model best captures the relationship between experience and salary. We will compare the models using the R^2 score and evaluate which one provides a better fit for the data.

> Question 1
>
> Lisez l'ensemble de données Salaire_données.csv et tracez un graphique pour visualiser la relation entre le salaire et l'expérience professionnelle. D'après le graphique, quelle est la nature de cette relation ?

We imported the libarary `pandas` so we can manage dataframe with it.

and we explored the dataframe using $df.head()$ and $df.columns$ and from that we know our dataframe have 100 rows and 2 columns and the name of the columns are $Exprecience, Salaire$

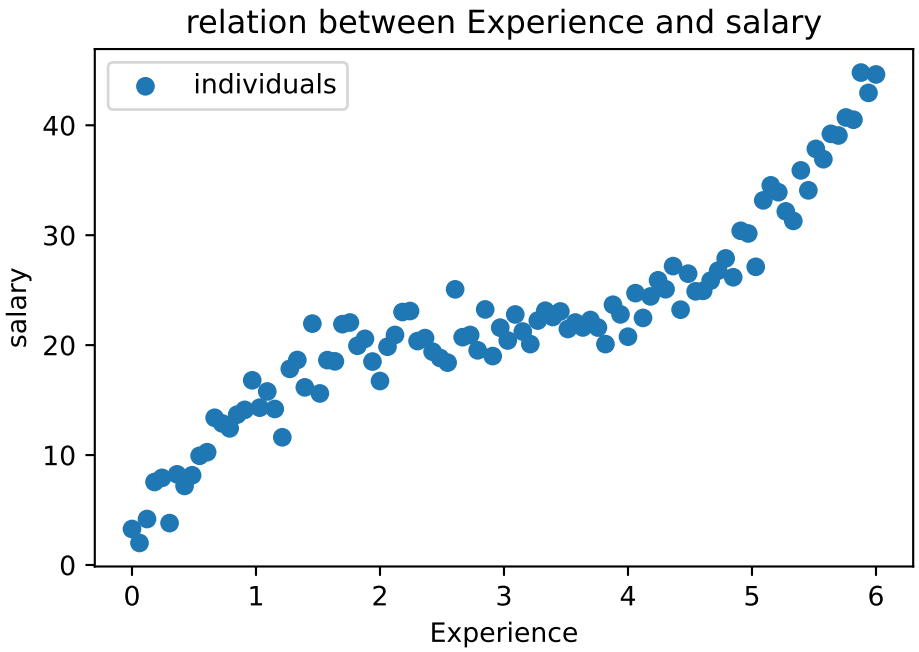
```
import pandas as pd
import numpy as np # we will use it later

df = pd.read_csv('data4.csv')

print(df.head())
print(df.shape)
```

```
   Experience   Salaire
0    0.000000  3.275294
1    0.060606  2.003359
2    0.121212  4.182534
3    0.181818  7.547732
```

```
4    0.242424  7.925572
(100, 2)
```

to plot the graph we need to import the library 'matplotlib' and as you can see the relation between *Exprecience* and *Salaire* aren't lineare so using a lineare model here wouldn't be ideal

```
import matplotlib.pyplot as plt

plt.scatter(df['Experience'],df['Salaire'], label = 'individuals')
plt.title("relation between Experience and salary")
plt.xlabel('Experience')
plt.ylabel('salary')
plt.legend()
plt.show()
```



Question 2

Séparez les données en variables explicatives X (années d'expérience) et en variable cible y (salaire). Ensuite, divisez l'ensemble de données en Training set et Test set en utilisant la fonction train_test_split, en prenant 30 % des données pour l'ensemble de test.

We splited our dataframe into two parts X for $Exprecience$ and Y for $Salaire$ and you can see that X is imported using [[]] so python can importe it as array so it can fit the model later.

we imported from $sklearn$ a function called $train_test_split$ it will helps to split the code for training and testing, and the parameter of this function the first two are the data you want to split in this case X, Y and the $test_size$ it's the proprtion of the test 0.3 is 30% of the data is for testing and rest is for training.

serve as idea of the shape of our data

```
X = df[['Experience']]
Y = df['Salaire']

from sklearn.model_selection import train_test_split

X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.3,random_state=7)

print(f"X_train shape: {X_train.shape}\n"
      f"X_test shape: {X_test.shape}\n"
      f"Y_train shape: {Y_train.shape}\n"
      f"Y_test shape: {Y_test.shape}")
```

```
X_train shape: (70, 1)
X_test shape: (30, 1)
Y_train shape: (70,)
Y_test shape: (30,)
```

1 Regression test

> Question 1
>
> Effectuez une régression linéaire pour prédire le salaire en fonction des années d'expérience.

to import Linear Regression model we need first to import $sklearn$ and then import $LinearRegression()$. after that we fit our data to the linear model using $fit.()$

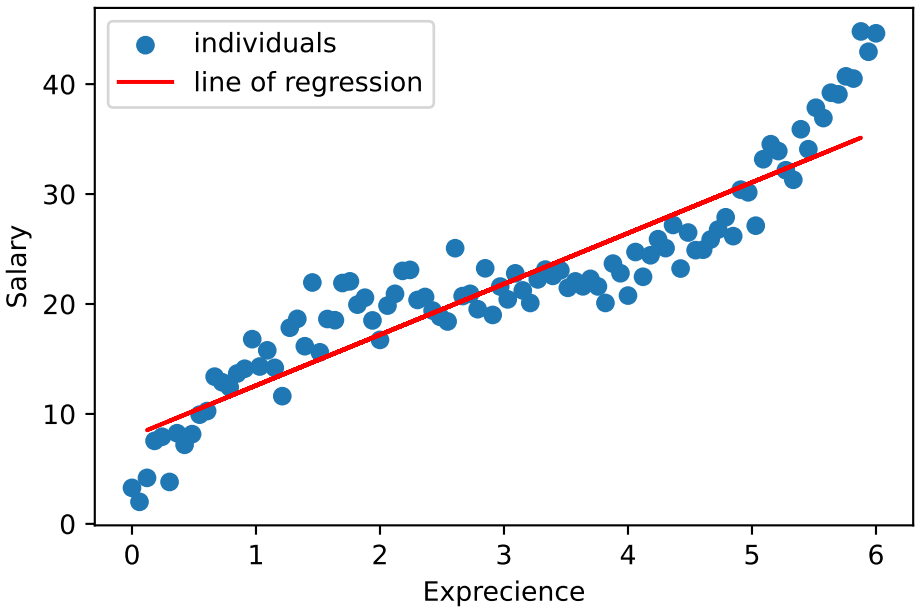
```
from sklearn.linear_model import LinearRegression

model_linear = LinearRegression()

model_linear.fit(X_train,Y_train)
```

```
LinearRegression()
```

> Question 2
>
> Tracez un graphique avec les points du dataset et la droite de la régression linéaire obtenue.

We simply plot the cloud of individuals and the line of regression using the library *matplotlib* it don't matter if we use the training data or the testing data because the coefficient and the Y-intercept are already calculted.

```
plt.scatter(df['Experience'],df['Salaire'],label='individuals')

plt.plot(X_test,model_linear.predict(X_test),color='red',label='line of regression')


plt.title("")
plt.xlabel('Exprecience')
plt.ylabel("Salary")
plt.legend()
plt.show()
```



Question 3

Utilisez r2_score de sklearn.metrics pour obtenir le score R^2 sur les prédictions de l'ensemble de test. Affichez le score pour voir dans quelle mesure le modèle explique la variance des données.

he R2 score, also known as the coefficient of determination, is a metric used to evaluate the performance of a regression model. To calculte the R2 score we need to import the sklearn library as the question said using this code $from sklearn.metrics import r2_score$.

To caluculte the R2 score you need to first calculte the residu which is the the real Y minus the predict Y , and then you calculte the total variance SS_{tot} as you can see the steps below:

$$R^2 = 1 - \frac{\text{variance of residuals}}{\text{total variance}}$$

Where:

$$\text{variance of residuals} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$\text{total variance} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

and the closet our model is to 1 the better it is and the close to 0 the worse it is, and the R^2 should be atleast above 0.7 if not you should add complexity or change the model or add more data.

```
from sklearn.metrics import r2_score

Y_pred = model_linear.predict(X_test)

r2_linear= r2_score(Y_test,Y_pred)

print("the R2 score is :",r2_linear)
```

```
the R2 score is : 0.8730934515217503
```

2 Appliquer la régression polynomiale

The math behind polynomial regression Polynomial regression is an extension of linear regression, where the relationship between the independent variable xx and the dependent variable yy is modeled as an nn-degree polynomial, and we write it in this form:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots + \beta_d x^d + \epsilon$$

and we calculte the coefficient with The normal equation:

$$\beta = (X^\top X)^{-1} X^\top \mathbf{y}$$

or with gradient decent, and we will explore more the polynomial regression.

> Question 1
>
> Utilisez **PolynomialFeatures** de **sklearn.preprocessing** pour créer des caractéristiques polynomiales de degré 4. Ensuite, appliquez cette transformation aux données d'entraînement et de test. Pourquoi cette transformation est-elle nécessaire avant d'appliquer une régression polynomiale ?

```
from sklearn.preprocessing import PolynomialFeatures

poly_transform = PolynomialFeatures(degree=4)

X_train_poly = poly_transform.fit_transform(X_train)
print('the shape of our new transformed data is:',X_train_poly.shape)
```

```
the shape of our new transformed data is: (70, 5)
```

As we can see the shape of our data was $(70, 2)$ and it is $(70, 5)$, This the new shape of our new data X it create 3 more columns of X^2, X^3, X^4

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 & x_1^4 \\ 1 & x_2 & x_2^2 & x_2^3 & x_2^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 & x_n^4 \end{bmatrix}$$

The transformation necessary before applying polynomial regression because without this tranformation the model cannot capture non-linear relationships between the variables.

Question 2

Entraînez un modèle de régression linéaire sur les données d'entraînement transformées et effectuez des prédictions sur l'ensemble de test. Quel est le score R^2 obtenu sur les prédictions de l'ensemble de test ?

here we repeated the same processing as before transforming our testing data to calculte $\hat{Y}$, and do the same thing as we did before to calculte the R^2.

```
model_linear2 = LinearRegression()
model_linear2.fit(X_train_poly, Y_train)

X_test_poly = poly_transform.fit_transform(X_test)
Y_pred_poly = model_linear2.predict(X_test_poly)

r2_poly=r2_score(Y_test,Y_pred_poly)

print("R2 score is:",r2_poly)
```

```
R2 score is: 0.9693139637728372
```

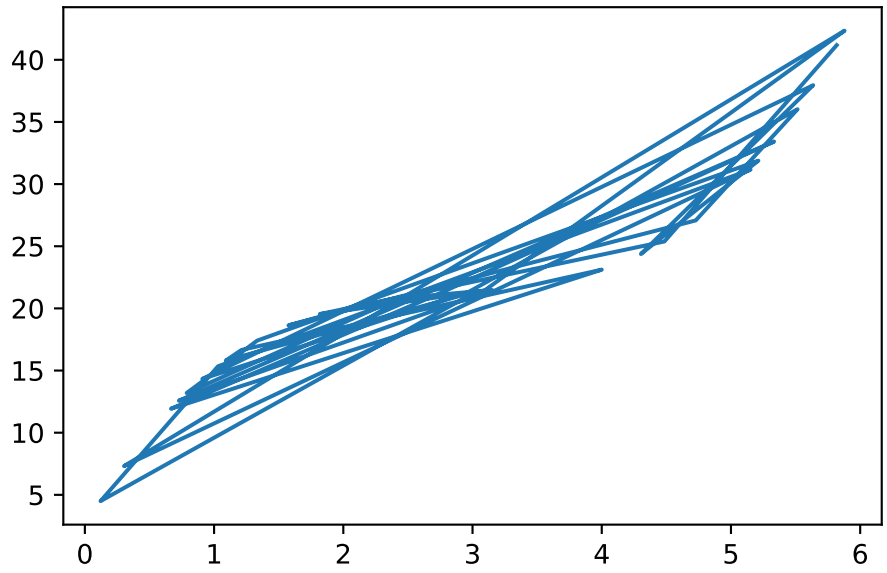
The $R^2 $ value of the polynomial model is closer to 1 compared to the $ R^2 $ value of the linear model. This indicates that the polynomial model provides a better fit to our data.

Question 3

Tracez un graphique avec les points du dataset et la courbe de régression polynomiale obtenue. Que pouvez-vous conclure de cette courbe par rapport à l'ajustement des données ?

To plot polynomial regression we can't do the same as linear regression because it's line let's say if we do the same what will apeare:

```
plt.plot(X_test,Y_pred_poly)
```



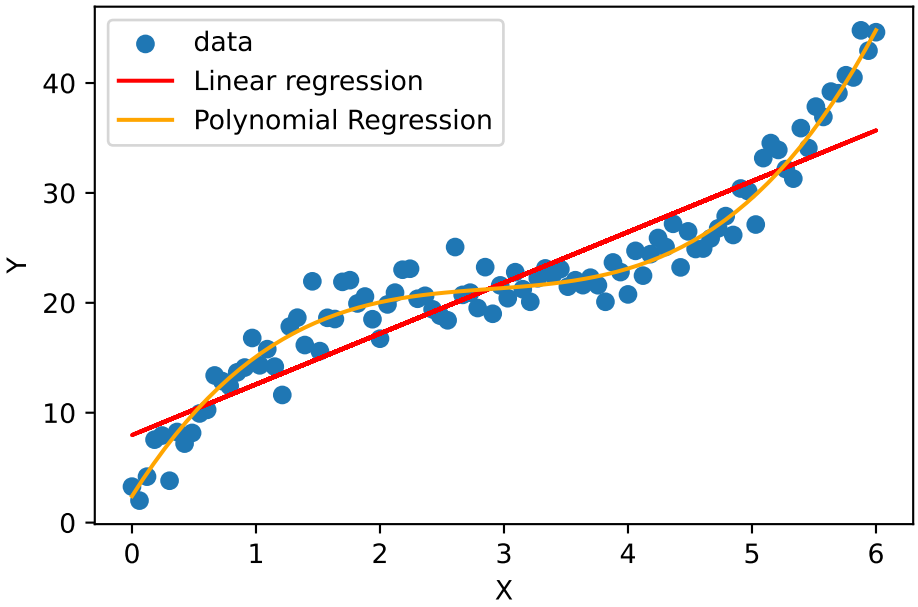
We will see that because plotting will attache each point to an other in order of each one in the array so it chaos, So we have to give to model values of the explicatve variable in ascending order.

We created a new variable X_range from 0 to 6 we got this values from the graph and we reshaped it to be a vector using $reshape(-1, 1)$ it's like transpose, After transforming the data, we used it to generate predictions.

```
X_range = np.linspace(0, 6, 70).reshape(-1,1)
X_range_poly = poly_transform.transform(X_range)
y_pred = model_linear2.predict(X_range_poly)

plt.scatter(X, Y,label='data')

plt.plot(X_train,model_linear.predict(X_train),color='red',label='Linear regression')

plt.plot(X_range, y_pred, color='orange', label='Polynomial Regression')


plt.xlabel('X')
plt.ylabel("Y")
plt.legend()

plt.show()
```

```
C:\Users\AL-Athir\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:4
```

```
warnings.warn(
```



> Question 4
>
> Estimez le salaire d'une personne ayant 4 ans d'expérience.

To estimate the salary of a person with 4 years of experience using a polynomial regression model of degree 4, the process is as follows:

1. **Transform the Input:** The input value 4 is transformed into its polynomial features, which include $4^1, 4^2, 4^3, 4^4$. These represent the experience raised to powers up to degree 4.
2. **Make the Prediction:** These transformed values are provided to the model, which uses the formula it learned during training to calculate the estimated salary. This is done by calling the `.predict()` method.

```
estima=np.array([[4]])

estima_transform=poly_transform.transform(estima)
estimated_values_for40=model_linear2.predict(estima_transform)
print('the estimated salary for person who had 4 years of exprerience is : ',estimated_values
```

```
the estimated salary for person who had 4 years of exprerience is :  [23.11600523]
```

```
C:\Users\AL-Athir\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:4
  warnings.warn(
```

Conclusion

The polynomial regression model provided a better fit for the data compared to the linear regression model. Since the relationship between salary and experience was non-linear, applying linear regression wouldn't have been ideal. The R2R2 score for the linear regression model was 0.83, while the polynomial regression model achieved a much higher R2R2 score of 0.97, indicating a much better fit. The predictions from the polynomial model were fairly accurate, with an R2R2 score close to 1. However, a limitation of the polynomial model is that it requires more calculations compared to linear regression.